



**SDJ** INTERNATIONAL  
COLLEGE - VESU

Affiliated To: Veer Narmad South Gujarat University, Surat  
Project Report

AS A PARTIAL REQUIREMENT FOR THE DEGREE OF  
*BACHELOR OF COMPUTER APPLICATIONS (BCA)*

BCASem V  
A.Y.2025-26

Project Report On Smart Expense Tracker

by

| Exam No. | Roll No. | Name             |
|----------|----------|------------------|
|          | 402      | Henny Viradiya   |
|          | 383      | Tanisha Vaghani  |
|          | 358      | Srushti Sutariya |
|          | 405      | Tisha Virani     |

Project Guide by:  
Prof . Himanshu Mishra

## Acknowledgement

The success and outcome of this project required a lot of guidance and assistance from many people, and we are extremely fortunate to have received this throughout the completion of our project work. Whatever we have done is only due to their guidance and assistance.

We would not forget to thank Principal Dr. Aditi Bhatt, IQAC coordinator and trust representative Dr. Vaibhav Desai, In-charge of PG Courses Dr. Vimal Vaiwala, In-charge of UG and Head of the UG Department Prof. Nainesh Gathiyawala and my project guide \_\_\_\_\_ and all other Assistant professors of SDJ International College Vesu, who took keen interest in our project work and guided us all along, till the completion of our project work by providing all the necessary information for developing a good system.

We are extremely grateful to her/him for providing such nice support and guidance, though she/he had a busy schedule managing the college affairs. We are thankful and fortunate enough to get support and guidance from all the Teaching staff of the IT Department, which helped us in completing our project work. Also, we would like to extend our sincere regards to all the non-teaching staff of the IT Department for their timely support.

|                         |           |
|-------------------------|-----------|
| <b>Henny Viradiya</b>   | (       ) |
| <b>Tanisha Vaghani</b>  | (       ) |
| <b>Srushti Sutariya</b> | (       ) |
| <b>Tisha Virani</b>     | (       ) |

# I N D E X

| Sr. No | Description                                  | Page No. |
|--------|----------------------------------------------|----------|
| 1      | Introduction                                 |          |
|        | 1.1 Project description                      |          |
|        | 1.2 Project Statement                        |          |
| 2      | System Requirements                          |          |
|        | 2.1 Software Requirements                    |          |
|        | 2.2 Hardware Requirements                    |          |
| 3      | System Design & Architecture                 |          |
|        | 3.1 PHP Version Architecture (Client-Server) |          |
|        | 3.2 React.js Version Architecture            |          |
|        | 3.3 Data Flow Diagram (DFD)                  |          |
| 4      | Technology Stack                             |          |
|        | 4.1 PHP Version (Server-Side Rendering)      |          |
|        | 4.2 React.js Version (Client-Side Rendering) |          |
| 5      | Implementation & Modules                     |          |
|        | 5.1 Login & Authentication                   |          |
|        | 5.2 New User Registration                    |          |
|        | 5.3 The User Dashboard                       |          |
|        | 5.4 Adding & Managing Expenses               |          |
|        | 5.5 The Admin Panel                          |          |
| 6      | Testing                                      |          |
| 7      | Conclusion                                   |          |
|        | Future Scope                                 |          |
| 8      | References                                   |          |
|        |                                              |          |

## **Table of Contents**

### **Summary**

### **Chapter 1: Introduction**

- 1.1 Project Overview
- 1.2 Problem Statement & Solution

### **Chapter 2: System Requirements**

- 2.1 Software Requirements
- 2.2 Hardware Requirements

### **Chapter 3: System Design & Architecture**

- 3.1 PHP Version Architecture (Client-Server)
- 3.2 React.js Version Architecture (Single Page Application)
- 3.3 Data Flow Diagram (DFD)

### **Chapter 4: Technology Stack**

- 4.1 PHP Version (Server-Side Rendering)
- 4.2 React.js Version (Client-Side Rendering)

### **Chapter 5: Implementation & Modules**

- 5.1 Login & Authentication
- 5.2 New User Registration
- 5.3 The User Dashboard
- 5.4 Adding & Managing Expenses
- 5.5 The Admin Panel

### **Chapter 6: Testing**

### **Chapter 7: Conclusion & Future Scope**

### **Chapter 8: References**

## Summary

"Greeny - Online Grocery Shop" is a e-commerce web application designed to offer a modern online grocery shopping experience for customers and efficient store management for administrators. Developed using PHP and MySQL, Node.js the project showcases the implementation of secure user authentication, dynamic product displays, robust shopping cart functionality, and a complete order processing system. This report details the project's scope, architecture, features, workflows, and technical documentation, providing a holistic view of the system.

## Chapter 1: Introduction

**1.1 Project Overview** This project is a web-based **Online Grocery shop Application**. The main goal of this application is to provide users with a simple and effective tool to order their daily needs. The application is designed to be intuitive and user-friendly, ensuring that even users with minimal effort can use it effectively.

The application is divided into two main modules:

1. **User Panel:** This is for individual users where they can create an account, log in, and order anything in grocery online from web app.
2. **Admin Panel:** This is a special panel for an administrator who can view and manage the data of all registered users.

### 1.2 Problem Statement & Solution

- **The Problem:**

In today's fast-paced world, many individuals struggle to find the time to visit physical grocery stores due to busy schedules, long queues, limited store hours. physical access to stores can be further disrupted, making the need for a reliable digital alternative more critical.

- **Our Solution:** The proposed solution is an Online Grocery Shop — a user-friendly web-based platform that enables customers to browse, search, and purchase groceries from the comfort of their homes.

## Chapter 2: System Requirements

### 2.1 Software Requirements

- **Operating System:** Windows, Linux, or macOS
- **Web Server:** Apache(forPHP version)
- **Backend/Frontend:** PHP7.4+, Node.js (for React version)
- **Database:** MySQL/MariaDB
- **Web Browser:** GoogleChrome, Mozilla Firefox, or any modern browser.

### 2.2 Hardware Requirements

- **Processor:** IntelCorei3 or equivalent
- **RAM:** 4 GB or more
- **Hard Disk:** 10GBoffree space

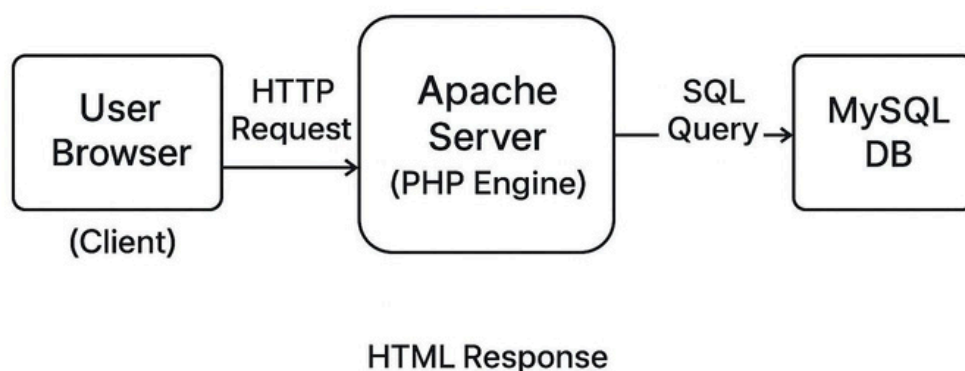
## Chapter 3: System Design & Architecture

### 3.1 PHP Version Architecture (Client-Server)

The PHP version follows a traditional multi-page application (MPA) architecture.

- **Client:** The user's web browser sends a request for a page.
- **Server:** The Apache server receives the request. The PHP engine processes the script, interacts with the MySQL database, generates a complete HTML page, and sends it back to the client.
- Every action (like submitting a form or clicking a link) results in a new page request to the server.

### Diagram :-

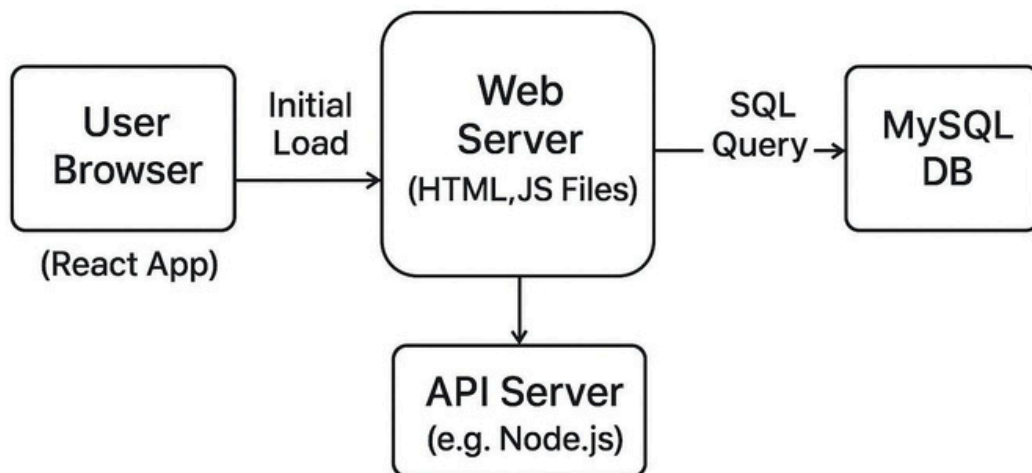


### 3.2 React.js Version Architecture (Single Page Application)

The React.js version is a modern Single Page Application (SPA).

- **Initial Load:** The server sends a single HTML file along with the JavaScript (React) code.
- **Client-Side Rendering:** React takes control of the page in the browser. When the user interacts with the app, React dynamically updates only the necessary parts of the page without a full page reload.
- **State Management:** Data is managed on the client-side. In a full-stack version, it would communicate with a backend API to fetch and send data.

#### Diagram :-



### 3.3 Data Flow Diagram (DFD)

This diagram is applicable to both versions as it represents the logical flow of data.

- **Level 0:** A user interacts with online grocery System. The system interacts with the Database.

- **Level 1 (User):** User logs in/registers, manages account, and views Cart. Data flows between the user, the system, and the database tables (**user**, **expense**).
- **Level 1 (Admin):** Admin logs in, views all data, and manages users.

## Chapter 4: Technology Stack

### 4.1 PHP Version (Server-Side Rendering)

- **Frontend:** HTML, CSS, Vanilla JavaScript (for small interactions).
- **Backend:** PHP (for all server-side logic and database communication).
- **Database:** MySQL.

### 4.2 React.js Version (Client-Side Rendering)

- **Frontend:** React.js (for building the user interface), Tailwind CSS.
- **State Management:** React Hooks (**useState**, **useContext**).
- **Backend (Simulated):** All data is currently stored and managed within the React application's state for this prototype.

## Chapter 5: Implementation & Modules

This chapter provides a look at the key features, which are functionally identical in both the PHP and React versions.

### 5.1 Login & Authentication

### 5.2 New User Registration

### 5.3 The User Dashboard

### 5.4 Adding & Managing Expenses

### 5.5 The Admin Panel



## 5.1 Login & Authentication

A secure entry point for the application allowing role-based access.

### Log In

**Email:**

**Password:**

Log In

Don't have an account? [Sign Up here.](#)

## 5.2 New User Registration

A simple form for new users to create an account.

### Sign Up

**Name:**

**Email:**

**Password:**

Register

Already have an account? [Log In here.](#)

### 5.3 The User Dashboard

The main landing page for a user, showing summary cards and quick actions.

Admin Panel

Dashboard Products Orders Customers

View Site

Logout

#### Dashboard

Today's Revenue

**3,245.00 Rs.**

Today's Orders

**0**

Pending Orders

**5**

Total Products

**53**

#### Recent Orders

| Order ID | Customer     | Total        | Status  | Action               |
|----------|--------------|--------------|---------|----------------------|
| #5       | srushti      | 310.00 Rs.   | Pending | <a href="#">View</a> |
| #4       | tisha virani | 2,235.00 Rs. | Pending | <a href="#">View</a> |

**5.4 Adding & Managing Expenses** The core functionality where Admin can perform CRUD (Create, Read, Update, Delete) operations on product manage

Admin Panel

Dashboard Products Orders Customers

View Site

Logout

#### Product Management

Add New Product

| ID | Name            | Final Price | Actions                                     |
|----|-----------------|-------------|---------------------------------------------|
| 76 | Watermelon      | 108.00 Rs.  | <a href="#">Edit</a> <a href="#">Delete</a> |
| 75 | Strawberry      | 237.50 Rs.  | <a href="#">Edit</a> <a href="#">Delete</a> |
| 74 | Pineapple       | 90.00 Rs.   | <a href="#">Edit</a> <a href="#">Delete</a> |
| 73 | Orange          | 90.00 Rs.   | <a href="#">Edit</a> <a href="#">Delete</a> |
| 72 | Lychee          | 135.00 Rs.  | <a href="#">Edit</a> <a href="#">Delete</a> |
| 71 | Guava           | 47.50 Rs.   | <a href="#">Edit</a> <a href="#">Delete</a> |
| 70 | Grapes          | 76.00 Rs.   | <a href="#">Edit</a> <a href="#">Delete</a> |
| 69 | Cinnamon powder | 270.00 Rs.  | <a href="#">Edit</a> <a href="#">Delete</a> |
| 68 | Turmeric powder | 114.00 Rs.  | <a href="#">Edit</a> <a href="#">Delete</a> |
| 67 | TATA Salt       | 135.00 Rs.  | <a href="#">Edit</a> <a href="#">Delete</a> |

## 5.5 The Admin Panel

A powerful overview of the entire system for the administrator, including viewing all users details

### Admin Panel

[Dashboard](#) [Products](#) [Orders](#) [Customers](#)[View Site](#)[Logout](#)

#### Order Management

| Order ID | Customer Name | Order Date                  | Total Amount | Status  | Actions                      |
|----------|---------------|-----------------------------|--------------|---------|------------------------------|
| 5        | srushti       | September 21, 2025, 7:44 pm | 310.00 Rs.   | Pending | <a href="#">View Details</a> |
| 4        | tisha virani  | August 22, 2025, 10:16 pm   | 2,235.00 Rs. | Pending | <a href="#">View Details</a> |

## Customer Reports

### Admin Panel

[Dashboard](#) [Products](#) [Orders](#) [Customers](#)[View Site](#)[Logout](#)

#### Customer Management

##### All Registered Customers

| CUSTOMER ID | NAME         | EMAIL             | ACTION                       |
|-------------|--------------|-------------------|------------------------------|
| 5           | srushti      | srushti@gmail.com | <a href="#">View History</a> |
| 2           | tisha virani | tisha@gmail.com   | <a href="#">View History</a> |

## Chapter 6: Testing

To ensure the application is reliable, robust, and bug-free, a comprehensive testing strategy was employed. We focused on different levels of testing to cover all aspects of the application, from individual functions to the complete user workflow.

- **Unit Testing:** In this phase, we tested the smallest individual parts of the code. For example, in the PHP version, we tested the database connection function separately to ensure it connects successfully. We also tested individual functions like the password hashing function during registration to confirm that it works as expected.
- **Integration Testing:** After unit testing, we tested how different modules work together. A key test case was the **Login-to-Dashboard flow**. We tested if a newly registered user could immediately log in and be redirected to the correct user dashboard, and not the admin panel. We also checked if an expense added by a user was correctly reflected in the admin's dashboard.
- **Validation Testing:** This was crucial for ensuring data integrity. We tested all input forms with various kinds of data:
- **Empty Submissions:** Checked if the system shows appropriate error messages when a user tries to submit an empty form.
- **Invalid Data Formats:** Tested the registration form with an invalid email format.
- **Data Type Mismatches:** Tried to enter text in the "price" field, which should only accept numbers. The system successfully handled all these cases by displaying user-friendly error messages.
- **Browser Compatibility:** The application was manually tested on the latest versions of Google Chrome and Mozilla Firefox to ensure that the layout, functionality, and styling are consistent across different browsers. We confirmed that the Tailwind CSS classes rendered correctly and the JavaScript for the charts and password toggles worked flawlessly on both platforms.

- **Testing Outcome:** All test cases were passed successfully. The application was found to be stable, secure, and performed as per the requirements. No critical bugs were found before deployment.

## Chapter 7: Conclusion & Future Scope

### 7.1 Conclusion

This project was a great learning experience. Building two versions using PHP and React.js provided deep insights into both server-side and client-side rendering paradigms. The final applications are practical and useful tools that can genuinely help people manage their finances more effectively. Both versions successfully achieve the project's objectives.

### 7.2 Future Scope

This project has a strong foundation and offers many possibilities for future enhancements to make it even more powerful and feature-rich.

#### 1. Voice & Chatbot Assistance

Implementing voice search and chatbot features (using tools like Google Assistant or custom AI chatbots) can enhance the user experience by making the platform more interactive and accessible.

#### 2. AI-based Product Recommendations

Using machine learning algorithms to suggest products based on user behavior, past purchases, or seasonal trends can increase customer engagement and boost sales.

ability for users to **export their reports as PDF or Excel files** for their personal records or for sharing.

- **Mobile App:** To provide users with easier access to their financial data on the go, a dedicated mobile application could be developed. This could be done using **React Native**, which would allow us to reuse some of the logic from our existing React.js web application.

## 8. References

### Website :-

**PHP Official Documentation:** <https://www.php.net>

**React.js Official Documentation:** <https://react.dev>

**MySQL Official Documentation:** <https://www.mysql.com>

**Tailwind CSS:** <https://tailwindcss.com>

**Chart.js for Data Visualization:** <https://www.chartjs.org>

**Stack Overflow Community:** <https://stackoverflow.com>

### **3.Subscription-based Models :**

Offering weekly or monthly subscription plans for essential groceries can ensure steady revenue and provide convenience to regular customers.

### **4.Mobile Application Development**

Creating a cross-platform mobile app (Android/iOS) would offer customers greater accessibility and a more convenient shopping experience on-the-go.

## **References**

PHP official Website

React js official Website

Node js official Website

My sql official Website

Stack Overflow